



KANN AUDITS

Security Review For **Mystic Finance**

mystic♦

Prepared For: **Mystic Finance**

Prepared By: **Kann, Shreyash, Volodymyr**

Date Reported: **May 11, 2026**

Audit Summary

Mystic Finance engaged Kann Audits to perform a manual security review of the Mystic Finance LST. From May 8 to May 10, a team of 3 auditors reviewed the in-scope source code. All findings identified during the assessment have been documented in the following report.

Protocol Overview

Mystic Finance is a liquid staking protocol built on Plume Network that allows users to stake PLUME and receive myPLUME, a yield-bearing liquid staking token representing their staked position. Validator rewards are automatically claimed and compounded, while users maintain liquidity through instant redemptions or the standard withdrawal process.

Scope

Repository: <https://github.com/mystic-finance/Liquid-Staking/tree/stake-plume-new>

Audited Commit: [75fe19a](#)

Final Commit: [75fe19a](#)

Files:

- [src/stPlumeMinter.sol](#)
- [src/Periphery/MyPlumeFeed.sol](#)

Methodology

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms, and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

The auditing process specifically considered:

- Testing against common and uncommon attack vectors
- Ensuring compliance with industry best practices and known smart contract security standards
- Verifying contract logic matches the client’s intended behavior
- Cross-referencing patterns with industry-standard implementations
- Manual line-by-line review by experienced auditors
- Access control verification
- Architecture and design analysis to ensure secure component interactions and clearly defined trust boundaries
- State transition and logic review to detect inconsistencies, unexpected behavior, or potential exploitation paths

Risk Classification

- **High** – result in directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
- **Medium** – result in moderate loss of funds or impact multiple users, but only under specific conditions. They are exploitable without privileged access but typically require certain assumptions or edge cases.
- **Low/Info** – issues are non-exploitable findings that do not pose a direct security risk or impact the system's integrity. These issues are typically cosmetic, best-practice deviations, or related to documentation and compliance. While they do not require urgent remediation, they may be addressed to improve code quality and maintainability.

Findings Count

High	Medium	Low/Info
0	1	4

Detailed Findings

ID	Title	Severity	Status
M-01	Calling <code>unstake()</code> Before Cooldown Expiry Resets Plume's Existing 21-Day Unlock Period for Previously Unstaked Funds	Medium	RESOLVED
L-01	Queued withdrawals can be frozen when a validator is deactivated without being slashed	Low	RESOLVED
L-02	Cooldown is over-delayed by one full batch interval when <code>unstake()</code> triggers batch processing	Low	RESOLVED
L-03	Explicit validator deposits bypass validator concentration caps	Info	ACKNOWLEDGED
L-04	<code>MyPlumeFeed.getMyPlumePri</code> oscillates by $\approx \textit{netReward}/\textit{totalSupply}$ across reward cycles due to a timing mismatch between numerator accrual and reward subtraction	Info	ACKNOWLEDGED

Calling `unstake()` Before Cooldown Expiry Resets Plume's Existing 21-Day Unlock Period for Previously Unstaked Funds

Medium

RESOLVED

Summary

The protocol batches validator unstakes through `_processBatchUnstake()` which internally calls:

```
plumeStaking.unstake(validatorId, amountToUnstake);
```

However, the underlying Plume staking system maintains a single cooldown position per validator. If `unstake()` is called again before the existing 21-day cooldown inside Plume finishes, the cooldown timer for the previously unstaked amount is reset together with the new unstake amount.

This causes already cooling funds to become relocked for another full cooldown duration.

Within the current implementation, additional queued withdrawals can trigger another batch unstake while a validator is still inside its active cooldown period:

```
if (
  totalQueuedWithdrawalsPerValidator[validatorId] >= withdrawalQueueThreshold ||
  block.timestamp >= nextBatchUnstakeTimePerValidator[validatorId]
) {
  _processBatchUnstake(validatorId);
}
```

As a result, newly queued withdrawals are merged into the existing validator cooldown position and restart the entire 21-day timer inside Plume for both old and new funds.

Impact

Users whose withdrawals are close to completion may unexpectedly have their cooldown reset back to 21 days due to later withdrawal activity.

Recommendation

Prevent additional batch unstakes from being executed while a validator already has an active cooldown in progress.

Instead of globally increasing `withdrawalQueueThreshold`, enforce a per-validator restriction during the cooldown window so queued withdrawals cannot retrigger another unstake before the current cooldown matures.

Proposed mitigation:

```
if (block.timestamp < nextBatchUnstakeTimePerValidator[validatorId]) {
  require(
    amount + totalQueuedWithdrawalsPerValidator[validatorId]
      < withdrawalQueueThreshold,
    "oversubscribed"
  );
}
```

Queued withdrawals can be frozen when a validator is deactivated without being slashed

Low

RESOLVED

Summary

Commit 75fe19a adds an `active` gate to the batch-unstake path in order to avoid calling `plumeStaking.unstake()` for slashed validators. However, the same change also blocks queued withdrawal processing for validators that are merely inactive but not slashed.

This matters because the upstream Plume staking layer distinguishes `inactive` from `slashed`:

- `slashed` validators are blocked for unstake operations
- validators that are only `inactive` are not slashed, and their stake remains recoverable

As a result, if users already have queued withdrawals on a validator and that validator is later deactivated, the queue can stop progressing even though the underlying Plume layer would still permit unstaking.

Root cause

The PR uses `getValidatorStats(...).active` as a proxy for “safe to process queued exits”.

That proxy is too strict.

For queued withdrawal liveness, the important condition is whether the validator is `slashed`, not whether it is currently `active`.

Relevant code

The regression is centered in two places inside `src/stPlumeMinter.sol`.

First, `processBatchUnstake()` now skips any validator whose `active` flag is false:

```
function processBatchUnstake() external {
    uint numVals = numValidators();
    uint256 index = 0;
    while (index < numVals) {
        uint16 validatorId = uint16(validators[index].validatorId);
        (bool active,,, ) = plumeStaking.getValidatorStats(validatorId);
        if (active && (totalQueuedWithdrawalsPerValidator[validatorId] >=
            withdrawalQueueThreshold || block.timestamp >=
            nextBatchUnstakeTimePerValidator[validatorId])) {
            _processBatchUnstake(validatorId);
        }
        index++;
    }
}
```

Second, `unstakeGov()` uses the same `active` dependency, so governance cannot use it to flush the stuck queue while the validator remains inactive:

```
function unstakeGov(uint16 validatorId, uint256 amount) external nonReentrant
    onlyByOwnGov returns (uint256 amountRestaked) {
    _rebalance();
    (bool active, ,uint256 stakedAmount,) = plumeStaking.getValidatorStats(uint16(
        validatorId));
    uint256 queued = totalQueuedWithdrawalsPerValidator[validatorId];
    uint256 total = amount + queued;
```

```
    if (active && stakedAmount > 0 && stakedAmount >= total) {  
        amountRestaked = plumeStaking.unstake(uint16(validatorId), total);  
        totalQueuedWithdrawalsPerValidator[validatorId] = 0;  
        nextBatchUnstakeTimePerValidator[validatorId] = block.timestamp +  
            batchUnstakeInterval;  
    }  
}
```

Attack / failure scenario

1. Users queue standard withdrawals against validator V.
2. Before the queued amount is processed, governance deactivates V without slashing it.
3. `totalQueuedWithdrawalsPerValidator[V]` remains non-zero.
4. `processBatchUnstake()` skips V because `active == false`.
5. `unstakeGov()` also cannot flush the queue because it is gated on the same `active` condition.
6. Pending withdrawals remain stuck until governance re-activates V.

Impact

This is a withdrawal liveness issue.

Affected users can be left waiting indefinitely for governance intervention even though the stake is not slashed and should still be recoverable.

Cooldown is over-delayed by one full batch interval when unstake triggers batch processing

Low

RESOLVED

Summary

When a user unstake request triggers `_processBatchUnstake`, the contract updates `nextBatchUnstakeTimePerValidator[validatorId]` to `block.timestamp + batchUnstakeInterval` before computing the user's cooldown timestamp. `_unstake` then derives `request.timestamp` from this updated “next batch schedule” value instead of the actual unstake initiation time. This introduces an unintended extra delay of `batchUnstakeInterval` to user withdrawals.

Root cause

The cooldown timestamp is derived from `nextBatchUnstakeTimePerValidator` after `_processBatchUnstake` mutates it for scheduling future batches.

Expected source for user cooldown: actual unstake initiation time (current transaction execution time). Actual source used: next scheduled batch time.

Technical details

In `_unstake` (single-validator branch):

- Queue is increased.
- If threshold/time condition is met, `_processBatchUnstake` is called.
- `_processBatchUnstake` sets `nextBatchUnstakeTimePerValidator = block.timestamp + batchUnstakeInterval`.
- `cooldownTimestamp` is set to `plumeStaking.getCooldownInterval() + nextBatchUnstakeTimePerValidator`.

Result: cooldown becomes `now + batchUnstakeInterval + externalCooldown`.

In `_unstake` (multi-validator branch):

The same pattern is repeated per validator, and `cooldownTimestamp` takes the maximum end time, where each end time uses the already-forwarded `nextBatchUnstakeTimePerValidator`.

Result: same unintended extra delay across all affected validators.

Why this is incorrect

`nextBatchUnstakeTimePerValidator` is a scheduling variable for future batch execution, not a record of when unstake was actually executed.

Using it as the basis for withdrawal cooldown conflates “next batch slot scheduling” with “actual unstake execution time”.

Impact

Users whose unstake requests trigger batch execution must wait approximately `batchUnstakeInterval` longer than intended by the underlying staking cooldown.

With defaults (`batchUnstakeInterval = 21 days + 1 hour`), this becomes a material lockup extension that significantly delays withdrawals and degrades protocol UX and trust.

POC

Assume `cooldownInterval = C` and `batchUnstakeInterval = B`.

At time `T`:

- User submits unstake and triggers `_processBatchUnstake`.
- External cooldown should end at `T + C`.
- Contract stores `request.timestamp = C + (T + B) = T + B + C`.
- `withdraw()` enforces `block.timestamp >= request.timestamp`.

Result: user is forced to wait an additional `B` time.

Recommendation

Do not derive request cooldown from `nextBatchUnstakeTimePerValidator`.

Instead, derive cooldown from actual batch unstake execution time (e.g. `block.timestamp` at `_processBatchUnstake`) or maintain a separate `lastBatchUnstakeTimePerValidator` updated only on execution.

Keep `nextBatchUnstakeTimePerValidator` strictly for scheduling logic.

Explicit validator deposits bypass validator concentration caps

Info

ACKNOWLEDGED

Summary

`getNextValidator()` enforces `maxValidatorPercentage` for auto-routed deposits. That check is reached only in the `_validatorId == 0` branch of `_depositEther()`.

When a user calls `submitForValidator(validatorId)` with an explicit `validatorId`, `_depositEther()` enters the explicit-validator branch. That branch only checks:

- validator activity
- validator capacity
- whether the full amount fits the validator's remaining capacity

It does not enforce the governance-configured `maxValidatorPercentage`.

Relevant code

```
if(_validatorId != 0){
    (uint256 validatorId, uint256 capacity) = _getValidatorInfo(_validatorId);
    (bool active, , , ) = plumeStaking.getValidatorStats(_validatorId);
    require(active, "Validator inactive");
    if(capacity > 0){
        require(_amount <= capacity, "Validator capacity exceeded");
        plumeStaking.stake{value: _amount}(uint16(validatorId));
        ...
    }
    require(remainingAmount == 0, "No capacity");
    return depositedAmount;
}
```

Impact

A user can route deposits directly to a validator and bypass the protocol's concentration cap enforcement.

This does not directly cause fund loss, but increases exposure to a single-validator slash beyond the governance-defined risk limits.

Proof of concept

- Validator 1 starts at 90 / 900 total stake.
- Governance caps validator 1 at 10%.
- Attacker calls `submitForValidator{value: 20 ether}(1)`.
- Deposit bypasses cap enforcement in explicit branch.
- Validator 1 exceeds configured concentration limit.

Recommendation

Apply the concentration check inside the explicit-validator branch before staking.

Ensure the same invariant enforced in auto-routing also applies to direct validator selection:

MyPlumeFeed.getMyPlumePrice() oscillates by $\approx \text{netReward} / \text{totalSupply}$ across every rewards cycle due to a timing mismatch between numerator accrual and the **getMyPlumeRewards()** subtraction

Info

ACKNOWLEDGED

Summary

MyPlumeFeed.getMyPlumePrice() is calculated as

```
getTotalDeposits() * 1e18 / frxETH.totalSupply().
```

getTotalDeposits() aggregates protocol-owned PLUME (**info.staked** + **info.cooled** + **info.parked** + **currentWithheldETH**) and subtracts a vested-rewards term:

```
getMyPlumeRewards() = rewardPerToken() * totalSupply() / 1e18
```

When rewards are processed via **loadRewards**, the net reward (**netReward** = **0.9 * reward**) is transferred back to **stPlumeMinter** and re-staked on the Plume diamond within the same transaction.

This amount is immediately reflected in **info.staked**, or **currentWithheldETH** for amounts below **minStake**.

However, **getMyPlumeRewards()** does not update instantly.

The Synthetix-style **updateReward** modifier in **stPlumeRewards** snapshots **rewardPerTokenStored** and **lastSync** before the new **rewardRate** is applied.

For the remainder of that block:

```
lastTimeRewardApplicable() - lastSync = 0
```

So **rewardPerToken()** returns the pre-load value.

As a result, the numerator increases by **netReward** immediately, while the subtractor remains unchanged.

This causes **getMyPlumePrice()** to spike by approximately **netReward / totalSupply()**.

Over the next **rewardsCycleLength** (7 days):

rewardPerToken() increases linearly, and **getMyPlumeRewards()** gradually catches up by the same **netReward**.

The price then decays back to its pre-load value at **rewardsCycleEnd**.

This creates a temporary pricing inconsistency across every reward cycle.

Root Cause

The issue comes from misuse of **getMyPlumeRewards()**.

It is a Synthetix-style reward accumulator.

These accumulators are designed to track rolling-vested rewards already registered in the system.

They are not intended to be used as balance-sheet accounting entries.

At the moment of **loadRewards**, the accumulator lags behind the actual fund inflow by one cycle.

`getTotalDeposits()` treats this accumulator as a liability and subtracts it from the numerator.

This only aligns correctly when:

`block.timestamp == rewardsCycleEnd`

At all other times, numerator and subtractor are out of sync, leading to oscillation in the computed price.

Disclaimers

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource, and expertise-bound effort in which we aim to identify as many issues as possible.

About Kann Audits

Kann Audits is a top-tier Web3 security audit company, trusted by leading projects and providing comprehensive audits and expert guidance to secure the most critical blockchain protocols.

Kann Audits provides services such as manual auditing, AI audits using the Kann AI Labs tool, and incident response.

To learn more, visit <https://kannaudits.com>

To view our audit portfolio, visit <https://github.com/KannAudits>

To book an audit, message <https://t.me/KannAudits>